v1.0.0

POWERCHAIN

WHITEPAPER

RENEWABLES • BLOCKCHAIN • TOKENIZATION

v1.0



POWERCHAIN COPILOT MOBILE

Technical Whitepaper & Product Documentation

Renewables • RWA operations • PWRC credits • Solana settlement

25 August 2026

Executive overview

PowerChain Copilot is a mobile AI operations interface for renewable infrastructure, real-world energy assets and onchain workflows. The application is designed around one principle: AI may coordinate work, but evidence, approvals, wallet signatures and settlement remain separate authoritative boundaries.

APP	PWRC	USAGE
v1.0.0	**9 decimals**	**10,000 PWRC**
Expo SDK 57	Token-2022	per response

Copilot interface Chat, prompt library and suggestions turn operational questions into structured research and analysis.	**Command Center** A mobile snapshot prioritizes verified energy, active assets, treasury state, alerts and next-best actions.
Review & Approve Evidence, provenance and risk are shown before approval. Approval still does not replace wallet signing.	**PWRC Credit Kernel** User-scoped integer accounting, deterministic quote hashes and append-only ledger history meter Copilot usage.

Authority boundary

The mobile client is not authoritative for telemetry, token balances, payment settlement, transaction success, evidence freshness or AI confidence. Those states must come from authenticated services and verified evidence.

What this document covers

- Mobile information architecture and UX principles
- RWA Copilot agent architecture and human execution controls
- PWRC credit accounting and HTTP 402 recovery
- Solana Pay top-ups and Payment Kernel verification
- Security, release controls and repository structure
- Visual design assets, mockups and implementation references

Mobile operator experience

The mobile application is intentionally not a dense blockchain dashboard. It is a decision surface for operators who need to understand what changed, what evidence supports it, what action is recommended and what authority is required next.

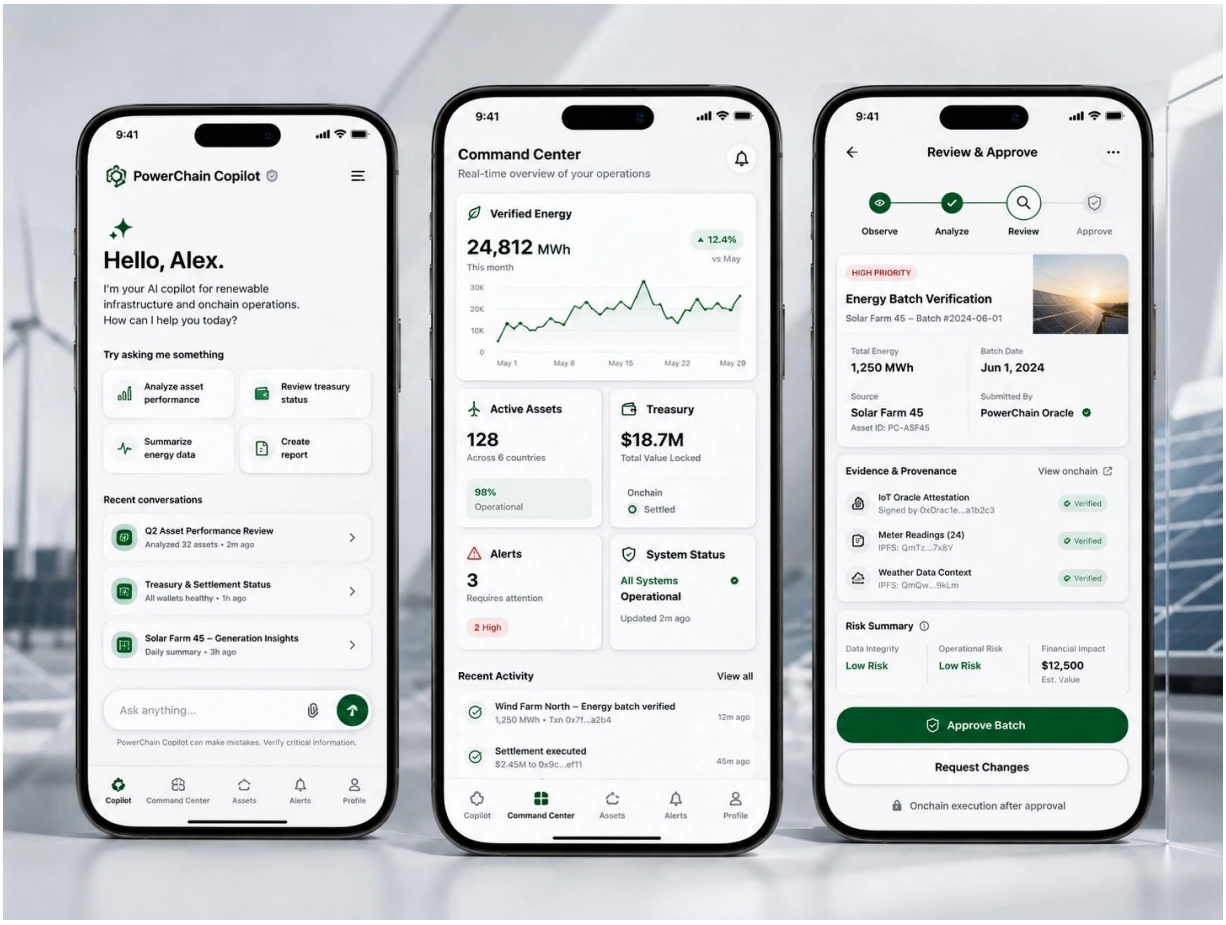


Figure 1. Copilot → Command Center → Review & Approve reference flow.

Copilot Conversational work surface with suggestions, recent threads, attachments and credit-aware execution.	**Command Center** Verified energy, asset health, treasury, alerts and recent activity in a single scan.
Assets Renewable facilities, meters, storage, RWA evidence and operational context.	**Alerts** Exceptions, stale evidence, policy violations and system degradation requiring attention.
Profile Identity, sessions, wallets, credits, network information and security controls.	**Review & Approve** Explicit human checkpoint between AI-prepared work and wallet-signable execution.

Command Center

Command Center is the operational home screen. It is optimized for short-session mobile use: system health first, the most important AI briefing next, then verifiable energy, asset, treasury and alert summaries.

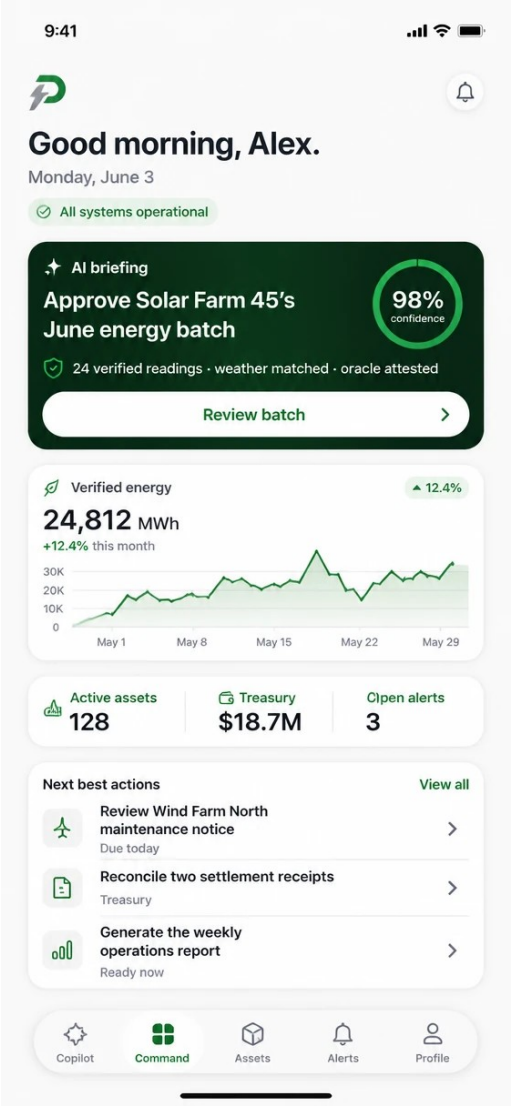


Figure 2. Reference Command Center layout emphasizing an actionable briefing and verified operational metrics.

Data presentation rule

Values such as MWh, treasury balances, confidence percentages, alert counts and settlement states must carry source identity and freshness. When the backend is unavailable, the UI must show unavailable or simulation state rather than plausible placeholders.

Recommended interaction states

- Loading: preserve layout and explain what source is being queried.
- Fresh: show source identity and last verified timestamp when material.
- Stale: retain the last verified value only with an explicit stale state.
- Unavailable: fail closed instead of substituting fabricated data.
- Simulation: visually separate modeled values from live evidence.

Onboarding and identity

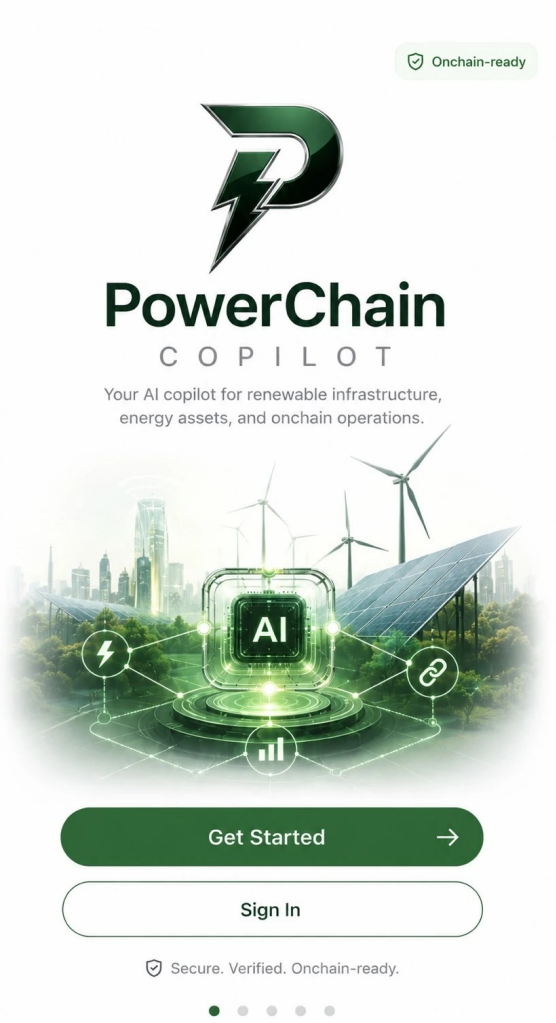


Figure 3. Onboarding reference: clear proposition, controlled calls to action and onchain-readiness language.

The onboarding experience introduces PowerChain Copilot as an AI assistant for renewable infrastructure, energy assets and onchain operations. “Onchain-ready” describes capability, not a guarantee that any specific operation is already settled or verified.

Identity sequence

1. Create or authenticate the user account.
2. Bind the active organization and authorized role.
3. Link one or more wallets through a signed challenge with replay protection.
4. Load user-scoped PWRC credit context and security posture.
5. Enter Copilot or Command Center with verified session context.

Security note

Wallet linkage proves control of a key for the challenge; it does not by itself authorize arbitrary financial execution. Execution remains intent-specific and requires the relevant approval and signature.

RWA Copilot architecture

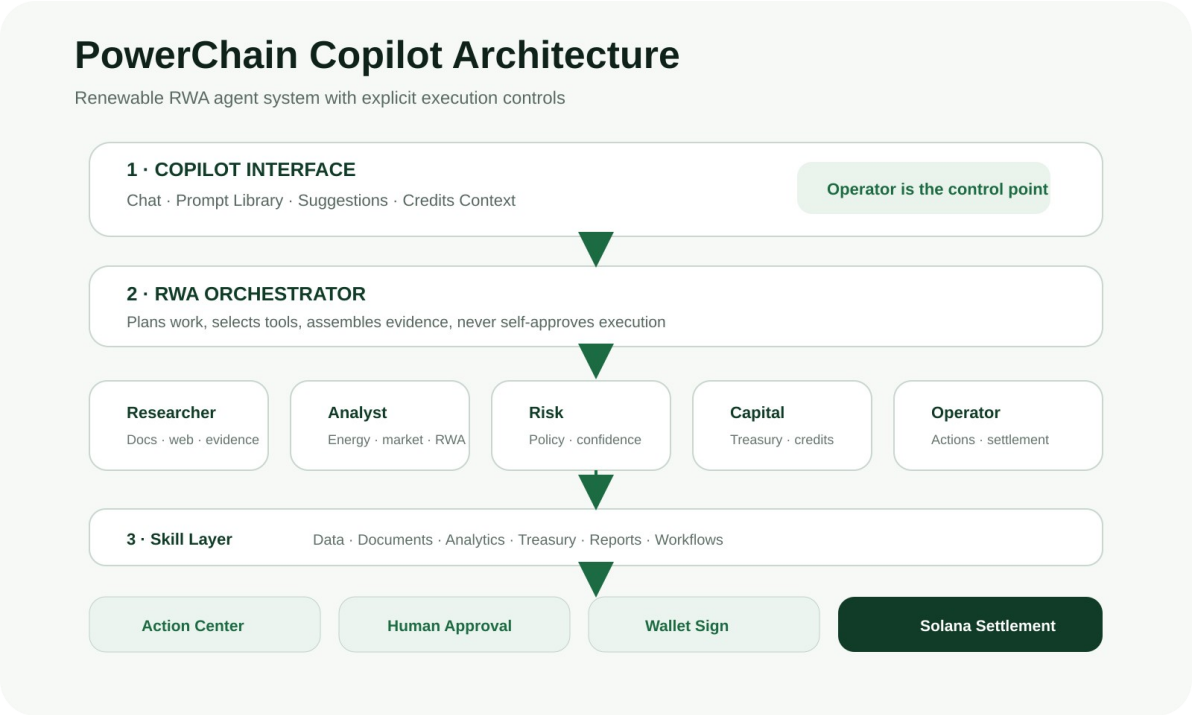


Figure 4. PowerChain RWA Copilot architecture with explicit human approval and wallet-signing boundaries.

Copilot is the operator interface. The RWA orchestrator plans work and coordinates specialized agents. Skills provide access to documents, analytics, treasury, reports and workflows. High-impact execution is separated into an Action Center, human approval and wallet signing.

Governance & safety

AI can research, analyze, coordinate and prepare. It does not silently move funds, approve itself or sign transactions.

Core agent roles

Researcher Retrieves documents, standards, market context and supporting evidence.	**Analyst** Computes renewable production, financial, portfolio and RWA analytics.
Risk Evaluates policy, evidence completeness, confidence and exception conditions.	**Capital** Prepares treasury, credit and funding context without bypassing approval.
Operator Builds controlled action intents and execution-review packages.	

Evidence-to-execution lifecycle

The system follows a deliberate execution sequence so generated text cannot be confused with operational authority.

6. Authenticate operator, organization, role and session.
7. Resolve the requested action and required evidence.
8. Collect evidence with source, provenance and freshness metadata.
9. Run policy and risk checks; surface missing evidence explicitly.
10. Simulate or preview material effects when applicable.
11. Require explicit human approval for high-impact actions.
12. Create a wallet-signable intent containing the exact final parameters.
13. Verify blockchain or provider settlement before reporting completion.

Physical authority Meter readings, device state and operational telemetry remain authoritative for physical energy claims.	**Financial authority** Treasury and settlement providers remain authoritative for balances and payment completion.
Blockchain authority Onchain confirmation proves the chain event, not offchain physical delivery unless linked evidence supports it.	**AI authority** AI provides analysis and coordination only; it does not manufacture evidence or settlement.

Fail closed

Any missing execution-critical source, secret, recipient, authorization, evidence requirement or verification step blocks the affected operation rather than degrading to a permissive mode.

PWRC Credit Kernel

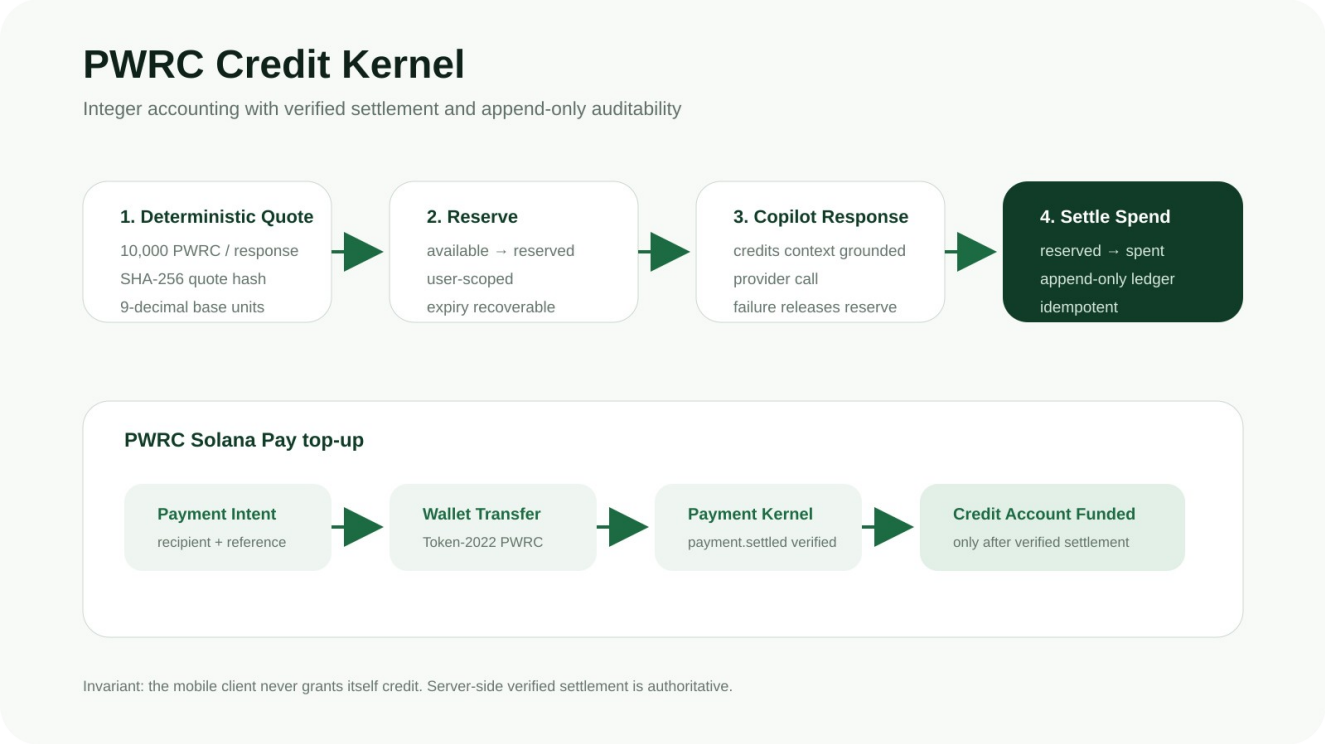


Figure 5. PWRC credit reservation, Copilot spend and verified Solana Pay funding.

1 PWRC	1 RESPONSE	DEMO GRANT
1,000,000,000	**10,000 PWRC**	**1,000,000 PWRC**
base units	deterministic price	100 responses

All credit math uses 9-decimal integer base units internally. JSON APIs expose decimal strings so JavaScript's 53-bit number limit cannot corrupt balances. Each user account tracks available, reserved, spent and funded balances with the invariant funded = available + reserved + spent.

Accounting controls

- Deterministic SHA-256 quote hashes bound to user, price version, quantity, amount and timestamps.
- Reservation before model execution prevents overspend under concurrency.
- Expired reservations restore availability without rewriting ledger history.
- Provider or orchestration failure releases the reservation.
- Successful responses settle reserved value into spent.
- Append-only ledger entries provide reconstructable balance history.
- Reconciliation compares current account snapshots with cumulative ledger deltas.

HTTP 402 and PWRC top-ups

When available PWRC credits cannot satisfy the deterministic quote, the metered Copilot endpoint returns HTTP 402 INSUFFICIENT_PWRC_CREDITS before model generation. The mobile UI shows the required and available amount and routes the operator to Credits.

402 response behavior

The user should never lose a generated answer because charging happened late. Pricing and reservation occur before provider execution, while provider failure releases reserved credit.

Solana Pay funding sequence

14. Create an authenticated, user-scoped PWRC top-up intent.
15. Bind the intent to the canonical PWRC mint, exact amount, treasury recipient and Solana Pay reference.
16. Open the Solana Pay URL in a compatible wallet.
17. Keep the mobile state pending; do not optimistically increment balance.
18. Verify payment.settled in the Payment Kernel with authenticated event evidence.
19. Bind recipient, reference, amount, user, network, mint and transaction evidence back to the original intent.
20. Append payment_credit ledger entry exactly once and refresh the authoritative account state.



Figure 6. PWRC brand reference used for the credit and payment experience.

Canonical PWRC identity

NETWORK	STANDARD	DECIMALS
Solana	**Token-2022**	**9**
mainnet-beta	Token Extensions	integer base units

Mint PWRCRXXZxbg6FdQZfK3PMD7KP8xfxs9acvifJiG46wc	**Program ID** TokenzQdBNbLqP5VEhdkAS6EPFLC1PHnBqCXEpPxuEb
Credit price 10,000 PWRC per completed Copilot response.	**Explorer** The mobile Credits screen exposes a direct Solana Explorer link to the canonical mint.

PWRC is used here as an application credit and supported payment/settlement asset. Product documentation must not imply that token ownership represents equity, debt, dividends or guaranteed returns. SaaS entitlement, network fees and token ownership should remain distinct concepts unless a product rule explicitly joins them.

Source of truth

The mobile source constant is src/constants/pwrc.ts. Deployment and documentation should fail review if the canonical mint, network, program or decimals diverge across surfaces.

Security architecture

The trust model is designed so a presentation-layer compromise cannot grant PWRC credits, fabricate payment settlement or silently authorize execution.

Authentication User identity is derived from the authenticated server context, not accepted as mutation authority from request bodies.	**Payment verification** Payment Kernel events are authenticated, replay-window checked and bound to the original recipient and reference.
Idempotency Retryable money-flow requests use idempotency keys, unique constraints and exactly-once accounting effects.	**Secret handling** Payment Kernel secrets, RPC credentials and private service keys belong server-side, never in Expo public environment variables.
Ledger integrity Credit ledger rows are append-only and reconciled against account snapshots.	**Execution safety** Policy → evidence → simulation/preview → approval → wallet sign → settlement verification.

Release security baseline

- Apply OWASP MASVS categories to local storage, authentication, networking, platform interaction, resilience and privacy.
- Use secure device storage for session material; avoid persistent plaintext secrets.
- Pin production API origins and reject unexpected environment fallback.
- Validate deep links and Solana Pay URLs before handing them to wallets.
- Capture audit IDs across client request, server action, payment intent and settlement verification.

Application implementation

EXPO SDK	REACT	ROUTING
57	19.2.3	Expo Router
RN 0.86	React Compiler	typed routes

The mobile application uses Expo SDK 57 with React Native 0.86, React 19.2 and typed Expo Router. The app remains a client of authenticated APIs; domain-critical accounting and settlement verification stay server-side.

src/app/ Route layouts and primary mobile surfaces.	**src/services/** Authenticated Copilot and Credits transport.
src/hooks/ UI orchestration for Copilot, theme and credit state.	**server/credits/** Quote, reservation, ledger, top-up and store abstractions.
server/payment-kernel/ Verified payment.settled boundary.	**server/copilot/** Credit-aware grounding and metered response orchestration.

Compatibility

Expo SDK 57 targets React Native 0.86 and React 19.2.x. Upgrade Expo and React Native as a compatible SDK line rather than independently mixing unsupported versions.

Documentation and visual system

The documentation site uses the same brand principles as the mobile UI: white and light-gray surfaces, dark green for trusted actions, black/dark ink for hierarchy, subtle borders and restrained depth. It avoids generic crypto neon effects, excessive gradients and decorative blockchain imagery.



Figure 7. PowerChain application icon treatment generated from the supplied brand mark.

Typography System sans-serif stack for UI and documentation; strong numerical hierarchy; no decorative display font dependency.	**Color** Dark green expresses primary actions and verified/trusted states; red/amber is reserved for actionable risk.
Radius Medium-to-large rounded corners support modern mobile surfaces without excessive glassmorphism.	**Motion** Short, functional transitions only. Respect reduced-motion preferences.
Icons Simple line icons with consistent stroke and semantic labels.	**Imagery** Prefer real product mockups, energy infrastructure and explanatory system diagrams over generic stock crypto visuals.

Documentation source structure

- apps/docs/ — static Next.js technical documentation application.
- shared/components/ui/docs/ — reusable document cards, callouts, figures and metrics.
- components/docs/ — composed documentation sections and navigation.
- shared/ui/logo.tsx and shared/ui/icons.tsx — shared brand and icon primitives.
- public/images/illustrations/ and public/images/mockups/ — optimized visual assets.
- public/assets/ — whitepaper DOCX/PDF and cover asset.
- references/ — token identity, architecture, design provenance and external standards.

References and release checklist

External references

- Expo SDK reference — https://docs.expo.dev/versions/latest/
- Expo project creation — https://docs.expo.dev/get-started/create-a-project/
- Solana Pay documentation — https://docs.solanapay.com/
- Solana Pay specification — https://github.com/solana-labs/solana-pay/blob/master/SPEC.md
- Solana Token Extensions / Token-2022 — https://solana.com/docs/tokens/extensions
- OWASP MASVS — https://mas.owasp.org/MASVS/

Release gate

- pnpm install completes without unintended package skew.
- pnpm test:credits passes all credit-kernel and API tests.
- pnpm check passes mobile lint, typecheck, credit tests and docs typecheck.
- pnpm doctor reports an Expo-compatible dependency graph.
- Production auth verifier and database transaction adapter are wired.
- PWRC_SOLANA_PAY_RECIPIENT and Payment Kernel secret are valid production values.
- Solana Pay verification is tested on the intended network and recipient.
- All mocked documentation values are clearly identified as design references, not live product evidence.
- PDF and documentation app are reviewed for clipping, accessibility and broken links.

Document status

This whitepaper documents the PowerChain Copilot mobile architecture and product implementation as of 25 August 2026. Runtime source and verified external systems remain authoritative where they differ from illustrative mockups.